

[illegible]

HARDWARE IMPLEMENTATION OF REAL TIME COMMUNICATION USING UART WITH AES 128 ENCRYPTION/DECRYPTION IN VERILOG HDL

Abstract:

This report gives details on the implementation of AES-128 bit encryption and decryption on Verilog Hardware Description Language and the modelling of real time communication between two NEXYS4 DDR FPGA boards. Advanced Encryption Standard(AES) is a symmetric block cipher defined in US Federal Information Processing Standard(FIPS). Encryption and Decryption has been an integral part of communication in the modern world, where a number of exchanges of valuable information take place through different private and public networks. Secure communication has become a crucial need for our technological infrastructures and this project intends to explore our industry standard security measures and explore low-cost embedded applications.

1. Introduction:

1.1 Project Motivation

Modern digital communication requires secure data transmission. This project explores industry-standard AES encryption implemented directly in hardware, offering:

- Real-time encryption/decryption at hardware speeds.
- Low-cost implementation on commodity FPGAs.
- Proven security - AES-128 remains unbroken since adoption.

1.2 System Overview

The system consists of three main components:

1. AES-128 Cipher - Symmetric block encryption.
2. UART Protocol - Asynchronous serial communication (9600 baud, 8-N-1).
3. Pipeline Architecture - Optimized for throughput.

Key Specifications

- Data block: 128 bits (16 bytes).
- Cipher key: 128 bits.
- Communication: UART at 9600 bps.

2. Advanced Encryption System 128

2.1. Overview:

AES is based on a design principle known as a substitution–permutation network. AES takes in data in blocks of 128 bits and encrypts them using the input 128 bit cipher key. The finite field operations are performed on two dimensional 4*4 array of bytes which is known as State.

→ 128 bits organized as 16 bytes.

Table 1: AES Transformation Method:

Each box represent 8 bit out of 128 bit													
in	in	in	in	→	S	S	S	S	→	out	out	out	out
00	04	08	12		00	04	08	12		00	04	08	12
in	in	in	in	→	S	S	S	S	→	out	out	out	out
01	05	09	13		01	05	09	13		01	05	09	13
in	in	in	in	→	S	S	S	S	→	out	out	out	out
02	06	10	14		02	06	10	14		02	06	10	14
in	in	in	in	→	S	S	S	S	→	out	out	out	out
03	07	11	15		03	07	11	15		03	07	11	15
Input				→	Process				→	Output			

The input plain-text is pasted into state as shown above and all operations are done into the state. The end state is then taken as output. Here's the main breakdown of all operations involved with AES Encryption and Decryption.

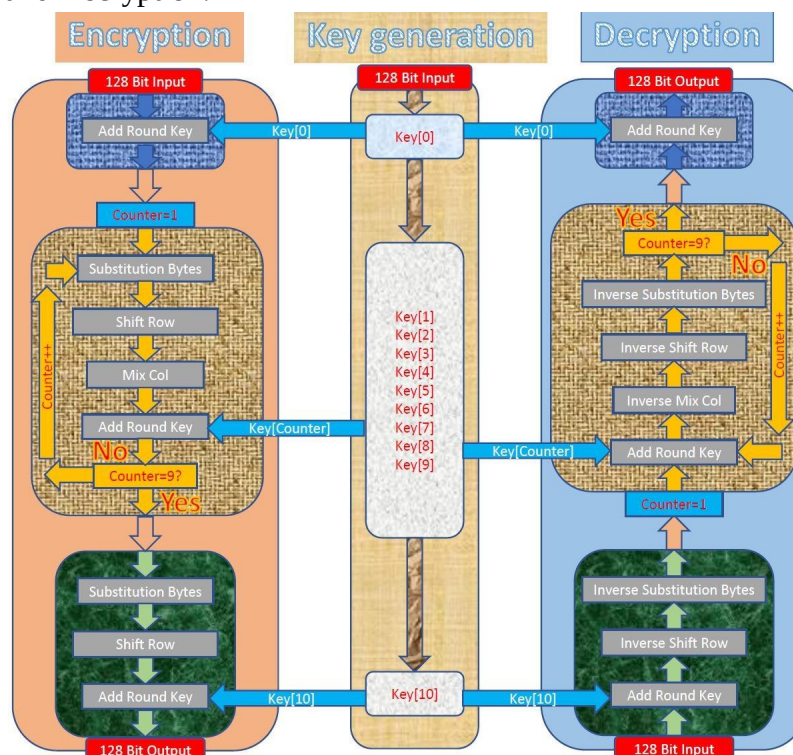


Figure 1: AES-128

2.2. Algorithm:

1. Encryption Algorithm

Initial: AddRoundKey(plaintext, K0)



Rounds 1-9: SubBytes → ShiftRows → MixColumns → AddRoundKey(K1-K9)



Round 10: SubBytes → ShiftRows → AddRoundKey(K10)



Ciphertext

2. Decryption Algorithm

Initial: AddRoundKey(ciphertext, K10)



Rounds 9-1: InvShiftRows → InvSubBytes → AddRoundKey(K9-K1) → InvMixColumns



Final: InvShiftRows → InvSubBytes → AddRoundKey(K0)



Plaintext

3. Key addition and inversion

3.1. Add Round Key

AES encryption is based on Galios Field(2^8). The add round key operation is performed by using bitwise operator.

The 128-Bit data and 128-Bit key are both arranged in 4×4 matrix for graphical representation.

Operation: Bitwise XOR between state and round key

Output[i] = State[i] $\oplus$ RoundKey[i]

Property: Self-inverse in $GF(2^8)$ - applying twice recovers original data.

Hardware: Simple XOR gates (128 parallel operations).

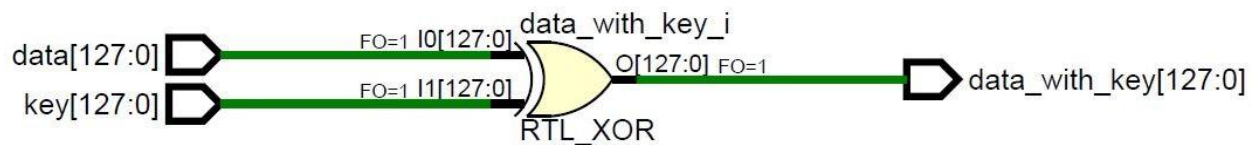


Figure 3: Add Round Key Schematics.

Table 2: Add Round Key:

D0	D4	D8	D12	L	K0	K4	K8	K12	=	A0	A4	A8	A12
D1	D5	D9	D13		K1	K5	K9	K13		A1	A5	A9	A13
D2	D6	D10	D14		K2	K6	K10	K14		A2	A6	A10	A14
D3	D7	D11	D15		K3	K7	K11	K15		A3	A7	A11	A15
Input Data					Key					Output			

3.2. Inverse Add Round Key

Inverse add round key operation is performed by using bitwise $\oplus$ operator between the cryptic data and the same key used to create the cryptic data from the original one.

• Pseudo Code for Add Round Key:

Original Data = Cryptic Data $\oplus$ 128-Bit Key

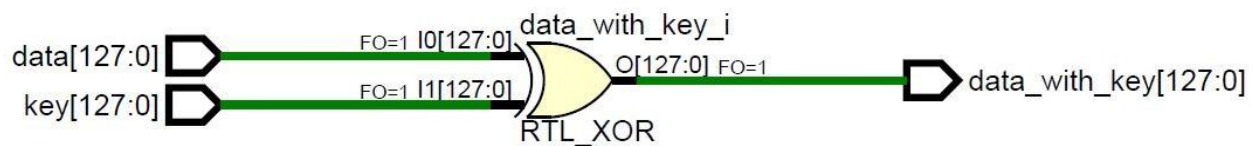


Figure 5: Inverse Add Round Key Schematics.

Table 3: Inverse Add Round Key

A0	A4	A8	A12	L	K0	K4	K8	K12	=	D0	D4	D8	D12
A1	A5	A9	A13		K1	K5	K9	K13		D1	D5	D9	D13
A2	A6	A10	A14		K2	K6	K10	K14		D2	D6	D10	D14
A3	A7	A11	A15		K3	K7	K11	K15		D3	D7	D11	D15
Cryptic Data					Key					Original Data			

4. Substitution Operation

4.1. Substitution bytes() Transformation

Operation: Non-linear byte substitution using Rijndael S-box

The S-box provides confusion by:

1. Computing multiplicative inverse in $GF(2^8)$.
2. Applying affine transformation with constant 0x63.

fig: AES-Substitution Box:

	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
--	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

0	63	7C	77	7B	F2	6 B	6F	C5	30	01	67	2B	FE	D7	A B	76
1	C A	82	C9	7D	F A	59	47	F0	A D	D4	A2	A F	9C	A4	72	C0
2	B7	F D	93	26	36	3F	F7	C C	34	A5	E5	F1	71	D8	31	15
3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	E B	27	B2	75
4	09	83	2C	1A	1B	6E	5 A	A0	52	3B	D6	B3	29	E3	2F	84
5	53	D 1	00	E D	20	F C	B 1	5B	6A	CB	BE	39	4A	4C	58	CF
6	D0	EF	A A	FB	43	4 D	33	85	45	F9	02	7F	50	3C	9F	A8
7	51	A 3	40	8F	92	9 D	38	F5	BC	B6	D A	21	10	FF	F3	D2
8	C D	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
9	60	81	4F	D C	22	2 A	90	88	46	EE	B8	14	D E	5E	0B	D B
a	E0	32	3A	0A	49	06	24	5C	C2	D3	A C	62	91	95	E4	79
b	E7	C8	37	6D	8 D	D 5	4E	A9	6C	56	F4	E A	65	7A	A E	08
c	B A	78	25	2E	1C	A 6	B 4	C6	E8	D D	74	1F	4B	B D	8B	8A
d	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
e	E1	F8	98	11	69	D 9	8E	94	9B	1E	87	E9	C E	55	28	DF
f	8C	A 1	89	0D	B F	E6	42	68	41	99	2D	0F	B0	54	B B	16

The S-box is constructed by performing the following transformation:

Calculate the multiplicative inverse in the finite field $GF(2^8)$ of the byte.

Apply the following transformation to the byte:

$$b'_i = b_i \oplus b_{(i+4) \bmod 8} \oplus b_{(i+5) \bmod 8} \oplus b_{(i+6) \bmod 8} \oplus b_{(i+7) \bmod 8} \oplus c_i$$

Here, b_i represents i 'th bit of the byte and c_i represents i 'th of a constant byte with a value of 63.

Example Mappings:

- $0x00 \rightarrow 0x63$
- $0x01 \rightarrow 0x7C$
- $0x59 \rightarrow 0xCB$
- $0xFF \rightarrow 0x1$

Table 4: Sub bytes () Transformation

A0	A4	A8	A12	S0	S4	S8	S12
A1	A5	A9	A13	S1	S5	S9	S13
A2	A6	A10	A14	S2	S6	S10	S14
A3	A7	A11	A15	S3	S7	S11	S15

4.2. Inverse Sub-bytes() Transformation

The Inverse Sub-bytes() Transformation is also a byte substitution operation performed on the individual byte of the 128-bit data during decryption. Uses inverse S-box (reverses the mapping), this also contains a mapping of each byte from 00 to FF and is inverse of the S-box.

fig: AES-Inverse Substitution Box

	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
0	52	09	6A	D 5	30	36	A5	38	B F	40	A3	9E	81	F3	D 7	FB
1	7C	E3	39	82	9B	2F	FF	87	34	8E	43	44	C4	D E	E9	C B
2	54	7B	94	32	A6	C2	23	3D	E E	4C	95	0B	42	FA	C3	4E
3	08	2E	A1	66	28	D9	24	B2	76	5B	A2	49	6D	8B	D 1	25
4	72	F8	F6	64	86	68	98	16	D 4	A4	5C	C C	5D	65	B6	92
5	6C	70	48	50	FD	ED	B9	D A	5E	15	46	57	A7	8D	9 D	84
6	90	D8	A B	00	8C	BC	D3	0A	F7	E4	58	05	B8	B3	45	06

7	D 0	2C	1E	8F	C A	3F	0F	02	C 1	AF	B D	03	01	13	8 A	6B
8	3 A	91	11	41	4F	67	D C	EA	97	F2	CF	C E	F0	B4	E6	73
9	96	A C	74	22	E7	A D	35	85	E2	F9	37	E8	1C	75	D F	6E
a	47	F1	1A	71	1D	29	C5	89	6F	B7	62	0E	A A	18	B E	1B
b	F C	56	3E	4 B	C6	D2	79	20	9 A	D B	C0	FE	78	C D	5 A	F4
c	1F	D D	A8	33	88	07	C7	31	B 1	12	10	59	27	80	E C	5F
d	60	51	7F	A 9	19	B5	4A	0D	2 D	E5	7A	9F	93	C9	9C	EF
e	A 0	E0	3B	4 D	A E	2A	F5	B0	C 8	EB	B B	3C	83	53	99	61
f	17	2B	04	7E	B A	77	D6	26	E1	69	14	63	55	21	0C	7D

Table 5: Inverse Sub bytes () Transformation

S0	S4	S8	S12	A0	A4	A8	A12
S1	S5	S9	S13	A1	A5	A9	A13
S2	S6	S10	S14	A2	A6	A10	A14
S3	S7	S11	S15	A3	A7	A11	A15

=

5. Shifting Operation

5.1. Shift-Rows() Transformation

Operation: Cyclically shift rows left by row index. The data are arranged in a 4 by 4 grid which is shown in the figure below. The following transformation is applied in each row[5]:

The first row is left unchanged.

The bytes in the second row are shifted one position to the left.

The bytes in the second row are shifted two positions to the left.

The bytes in the second row are shifted three positions to the left.

The array of bytes before and after transformation is shown below:

Table 6: Shift Rows () Transformation

S0	S4	S8	S12		S0	S4	S8	S12
S1	S5	S9	S13		S5	S9	S13	S1
S2	S6	S10	S14		S10	S14	S2	S6
S3	S7	S11	S15	=	S15	S3	S7	S11

5.2. Inverse Shift-Rows() Transformation

Shift right instead of left.

The first row remains unchanged.

The bytes in the second row are shifted one position to the right.

The bytes in the second row are shifted two positions to the right.

The bytes in the second row are shifted three positions to the right.

The array of bytes before and after transformation is shown below:

Table 7: Inverse Shift Rows () Transformation

S0	S4	S8	S12		S0	S4	S8	S12
S1	S5	S9	S13		S13	S1	S5	S9
S2	S6	S10	S14		S10	S14	S2	S6
S3	S7	S11	S15	=	S7	S11	S15	S3

This is for the first column, and the pattern is similar for all the other columns as shown in figure above.

6. Column Operations

6.1. Mix Column

Operation: Matrix multiplication in $GF(2^8)$.

Each column is multiplied by the MixColumns matrix.

After the input data is arranged in 4×4 matrix, it undergoes matrix multiplication with the Mix Col Matrix.

Table 9: Mix Column Operation

Mix Col Matrix					Input Data Matrix					Output Data Matrix			
02	03	01	01	×	D0	D4	D8	D12	=	O0	O4	O8	O12
01	02	03	01		D1	D5	D9	D13		O1	O5	O9	O13
01	01	02	03		D2	D6	D10	D14		O2	O6	O10	O14
01	01	02	03		D3	D7	D11	D15		O3	O7	O11	O15

Column Wise multiplication is visualized in table 10.

Table 10: Column wise multiplication in Mix Col Operation

Mix Col Matrix				×	Input Col		Output Col
02	03	01	01		D_n		O_n
01	02	03	01		D_{n+1}	=	O_{n+1}
01	01	02	03		D_{n+2}		O_{n+2}
01	01	02	03		D_{n+3}		O_{n+3}
$n \in \{0,4,8,12\}$							

The matrix multiplication only involves multiplication by 01,02 and 03 in Galios Field(2^8) .

Galois Field Arithmetic:

- Addition: XOR operation.
- Multiplication by 0x01: Identity (output = input).
- Multiplication by 0x02: Left shift with conditional XOR by 0x1B if overflow.
- Multiplication by 0x03: (input $\times$ 2) $\oplus$ input.

Irreducible Polynomial: $x^8 + x^4 + x^3 + x + 1$ (0x11B).

Hence, Mix Column operation is conducted in $GF(2^8)$ feild using the above principle of multiplication and addition.

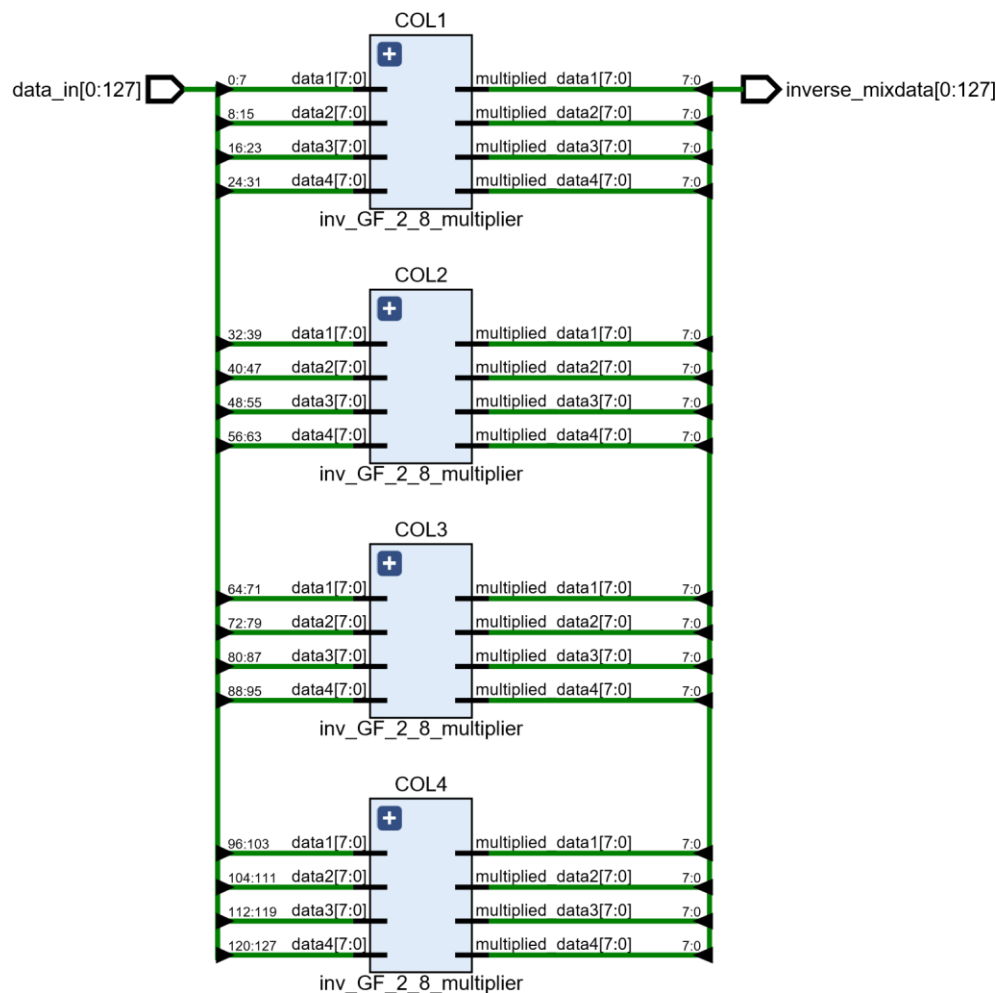


Figure 17: Schematic View of Mix Columns

6.2. Inverse Mix Column

→ uses inverse matrix.

Table 14: Inv Mix Column Operation

Inv Mix Col Matrix				Input Data Matrix				Output Data Matrix			
0E	0B	0D	09	D0	D4	D8	D12	O0	O4	O8	O12
09	0E	0B	0D	D1	D5	D9	D13	O1	O5	O9	O13
0D	09	0B	0D	D2	D6	D10	D14	O2	O6	O10	O14
0B	0D	09	0E	D3	D7	D11	D15	O3	O7	O11	O15

The matrix multiplication only involves multiplication by 0E, 0B, 0D, and 09 in Galois Field(2^8).

Multiplication by 09 input $\times$ 09 = input $\times$ (00001001) = input $\times$ ($1000 \oplus 0001$) = (input $\times$ 1000) $\oplus$ (input $\times$ 0001)

input $\times$ 1000 = input $\times$ (10) $\times$ (10) $\times$ (10) = input $\times$ 2 $\times$ 2 $\times$ 2

$\text{input} \times 0001 = \text{input}$

Hence,

$\text{input} \times 09 = (\text{input} \times 2 \times 2 \times 2) \oplus (\text{input})$.

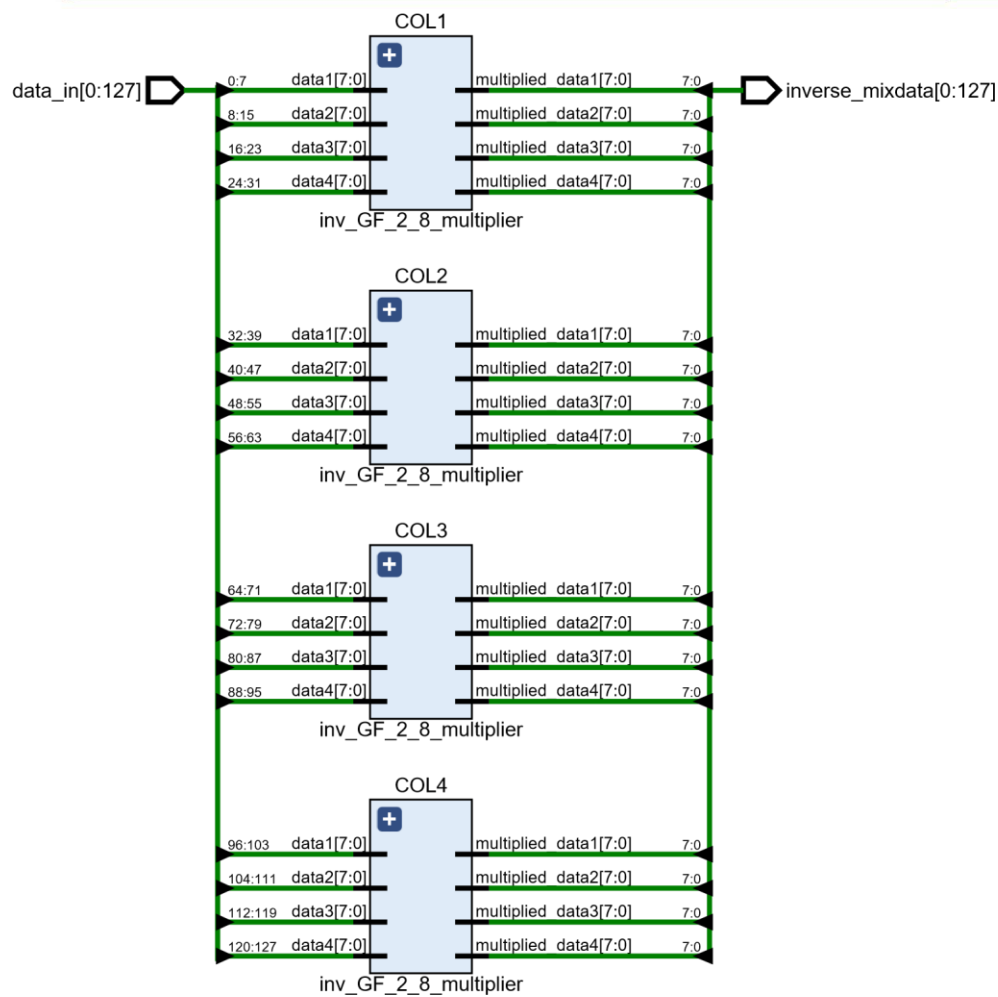
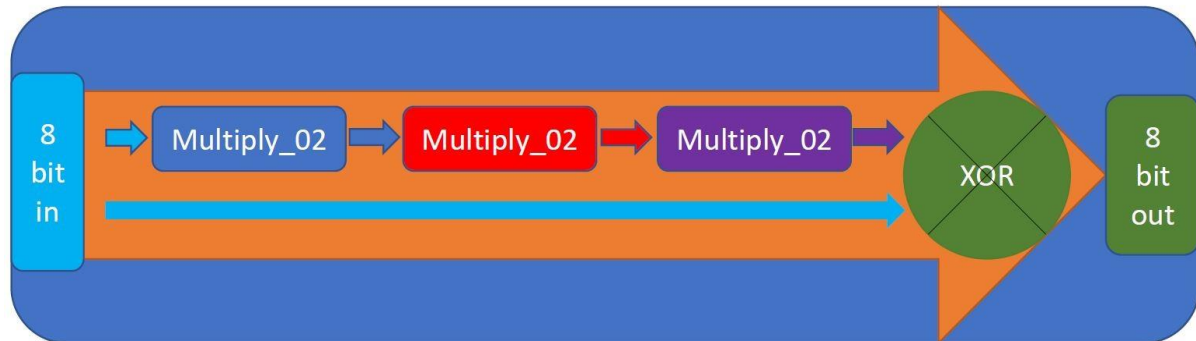


Figure 23: Schematic View of Inverse Mix Columns
7. Key Expansion

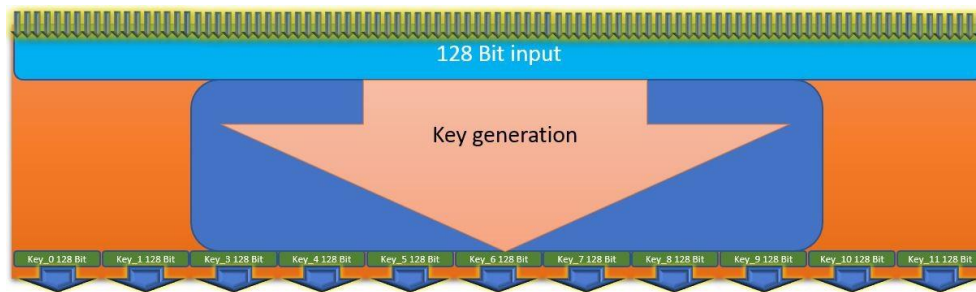


Figure 24: Key Generation pseudo circuit

Overview : Round Keys required for each AddRoundKey iteration is derived from a single Cipher Key by the means of the KeyExpansion algorithm.

Purpose: Generate 11 round keys (K0-K10) from single 128-bit cipher key.

Algorithm:

1. Initial key split into 4 words: $w[0]$, $w[1]$, $w[2]$, $w[3]$
2. For rounds 1-10:
 - Take previous last word (e.g., $w[3]$).
 - Apply **RotWord** (circular byte shift).
 - Apply **SubWord** (S-box substitution).
 - XOR with **Rcon** (round constant).
 - XOR result with previous round's first word.

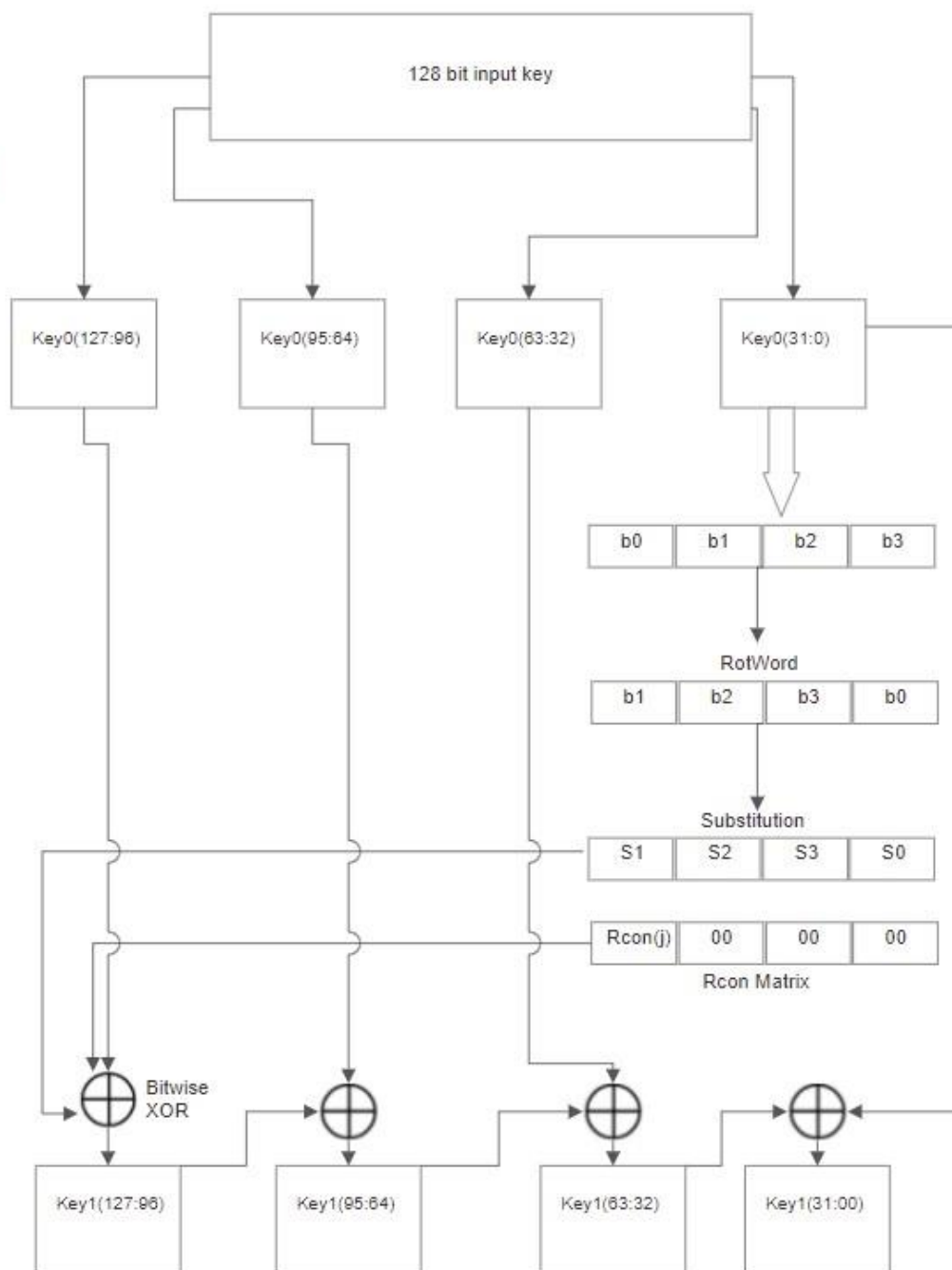
Figure 26: Round Constant(RCON)

where multiplication is defined in the Galius Field ($GF(2^8)$). [3]

Round	1	2	3	4	5	6	7	8	9	10
Hex	01	02	04	08	10	20	40	80	1b	36
Dec	01	02	04	08	16	32	64	128	27	54

Generated via: $Rcon[i] = 0x02^{(i-1)} \bmod (x^8 + x^4 + x^3 + x + 1)$.

7.2. Key Expansion in Pictures



8. UART:

8.1. UART for 8 bit Data Transfer

Two UARTs directly communicate with each others using two wires for their transmitting and receiving ends.

The two signals of each UART device are named Transmitter(Tx) and Receiver(Rx). The main motive of a transmitter and receiver line for each device is to transmit and receive serial data intended for serial communication. The UART transmitter and receiver have to agree on some parameters, such as:

Baud Rate: 9600

Number of data bits: 8

Parity Bit : 0

Stop Bits: 1

Flow Control: None

Baud rate: The frequency at which the receiver captures the incoming bits and colloquially, the frequency at which the transmitter sends the outgoing bits.

The number of data bits: It is set to eight bits at a time. That means, in each round, a byte is sent over the channel.

Parity bit: It is to check the validity of the transmitted data over UART, and is appended after the data is sent which is calculated by doing an XOR operation on all of the bits.

Stop Bits: A stop is always set is 1, and it indicates the end of transmission of a byte.

Flow Control: It is likely to be set to None, and is not widely used in present days applications.

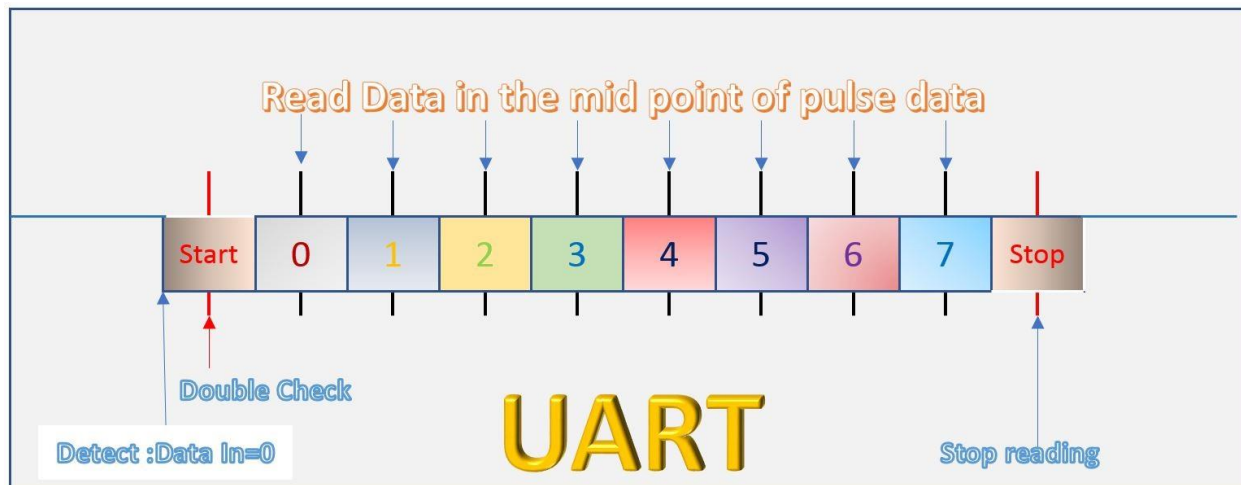


Figure 29: UART

4.3 Timing Calculations

Clock Cycles per Bit:

100 MHz FPGA Clock ÷ 9600 baud = 10,417 clock cycles/bit.

Sampling Point: Middle of bit period (5,208 cycles after start bit edge)

Working Mechanism:

Uart receives parallel data from the data bus and creates a serial packet and adds a start bit stop bit and a parity bit.

Start bit pulls UART line to 0 (usually kept at 1), which indicates the receiver to capture incoming data while stop bit will stop communication by pulling the line back to 1.

The receiver will ignore start, stop, and parity bits and captures data into the parallel stream, and sends it to the data buses.

Now, we will talk about the UART transmitter and UART receiver in details.

9. UART 128 bits

9.1. Baud Rate:

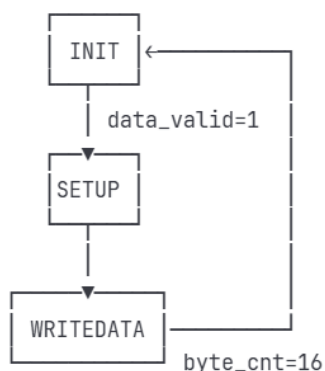
Since UART transmits 8 bits at a time:

- **128-bit block** = 16 sequential UART frames
- **Total time** = 16 bytes $\times$ (1 start + 8 data + 1 stop) bits $\times$ (1/9600 s/bit) $\approx$ **16.7 ms per block**

Table 18: Input/Output wire for Uart Transmitter

I/O	Wire	Size(bit)	Utilization
Input	clock	1	Clock for operations
	data	128	data that needs to be sent
	data state	1	Data _ state is a flag that initiates the transmitter module.
	reset	1	Resets the transmitter to state Init.
Output	data out tx	1	Output pin.

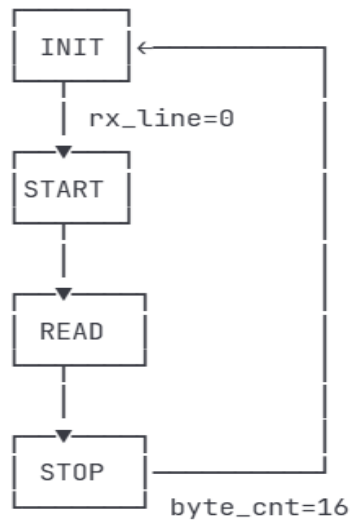
9.2 Transmitter State Machine:



States:

- **INIT:** Wait for data_valid flag.
- **SETUP:** Pack 8 bits with start/stop bits.
- **WRITEDATA:** Transmit 10-bit frames, loop until 16 bytes sent.

9.3 Receiver State Machine:



States:

- **INIT:** Wait for start bit (line goes low).
- **START:** Confirm start bit at mid-point.
- **READ:** Sample 8 data bits at baud rate intervals.
- **STOP:** Verify stop bit, output 128-bit block when complete.

10. Pipeline Architecture

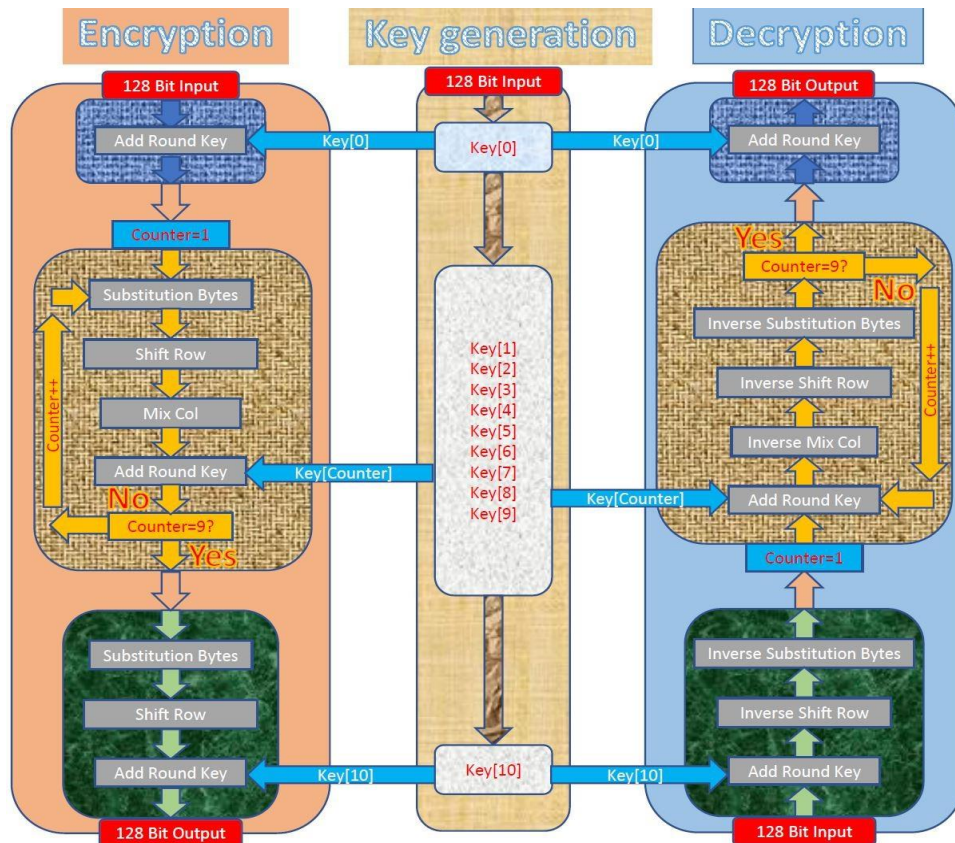


Figure 42: Full AES Encryption, Decryption and Key Generation.

10.1. Encryption Pipeline:

This is the encryption process. Key generation is a circuit without registers so they are ignored. Appropriate keys are passed at appropriate circuits.

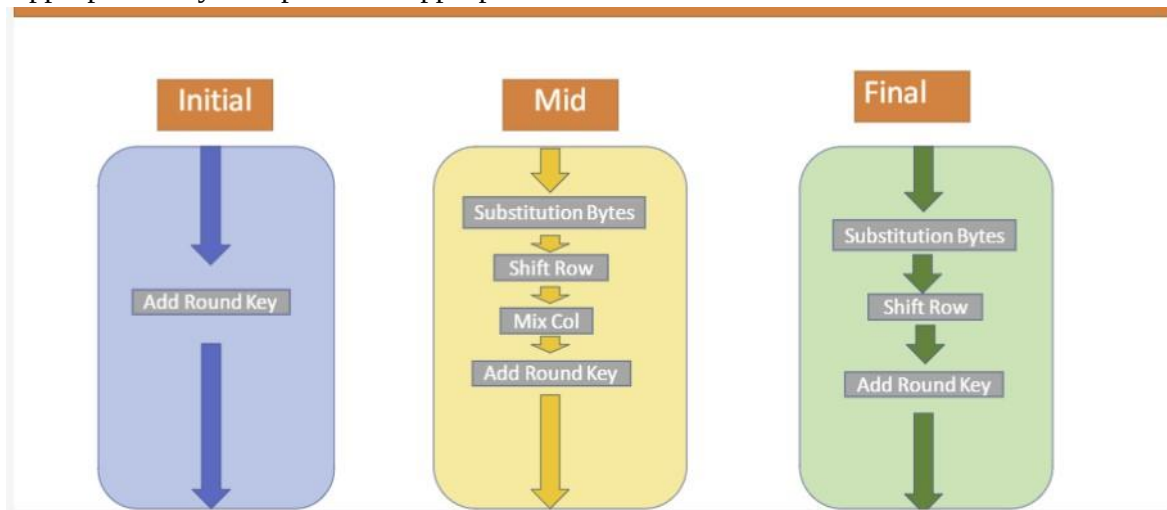
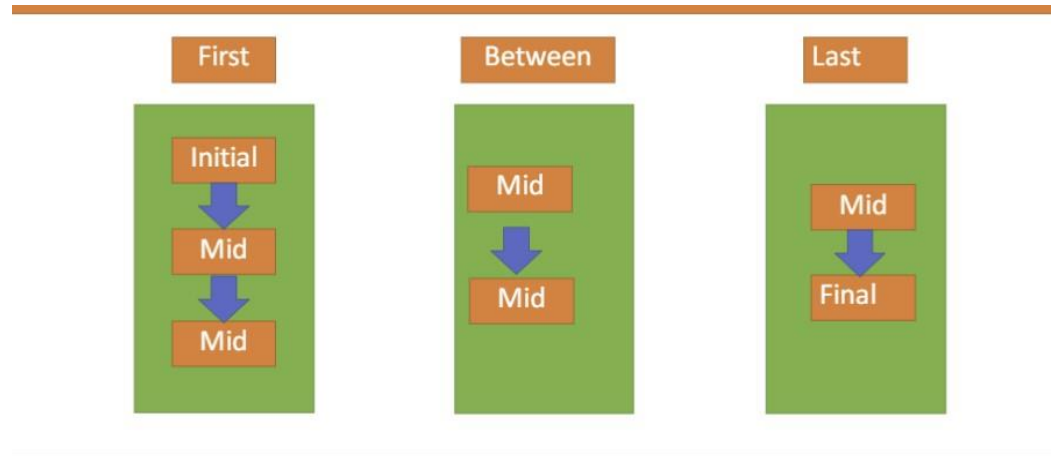


Figure 44: Encryption Broken down to different parts that can be in turn implemented to state machine.

10.1.2. Three circuits for Encryption:

Each of these circuit take in 128 bit data and output 128 bit data. Combination of these three circuits represents complete encryption. The circuit First has three parts as the initial circuit has only add roundkey whilst Between circuit and Final Circuit are more complex.



Stage 1 (FIRST):

- AddRoundKey(plaintext, K0)

Stage 2 (BETWEEN):

- 9 iterations of: SubBytes → ShiftRows → MixColumns → AddRoundKey(K1 -K9).

Stage 3 (LAST):

- SubBytes → ShiftRows → AddRoundKey(K10).

10.2. Decryption Pipeline:

This is the decryption process. Key generation is a circuit without registers so they are ignored. Appropriate keys are passed at appropriate circuits.

These three chunks as combined to form three circuits as shown in figure.

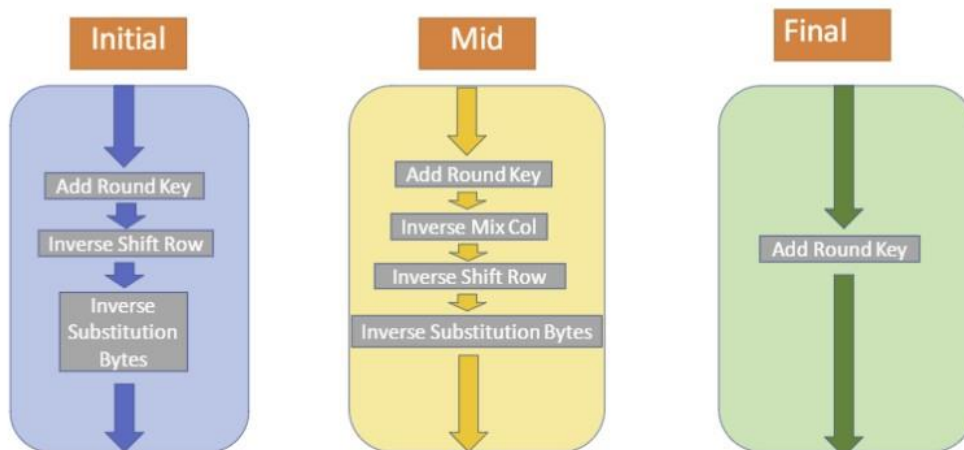
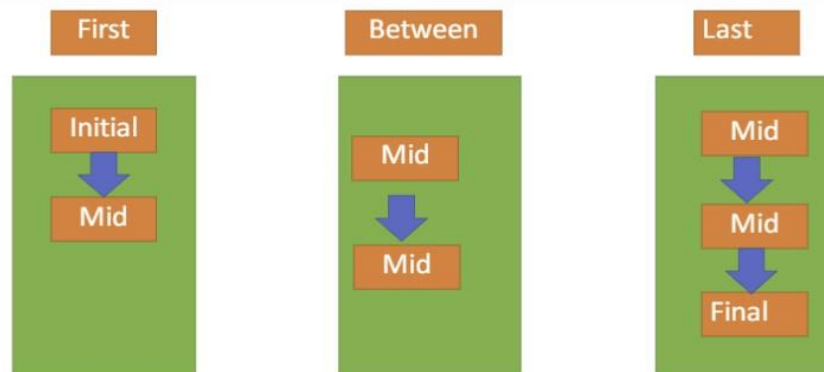


Figure 49: Decryption broken down to different parts that can be in turn implemented to state machine.

10.2.2. Three circuits for Decryption

Each of these circuit take in 128 bit data and output 128 bit data. Combination of these three circuits represents complete encryption. The circuit Last has three parts as the final circuit has only add roundkey whilst between circuit and First Circuit are more complex.



Stage 1 (LAST):

- AddRoundKey(ciphertext, K10)..

Stage 2 (BETWEEN):

- 9 iterations of: InvShiftRows → InvSubBytes → AddRoundKey → InvMixColumns.

Stage 3 (FIRST):

- InvShiftRows → InvSubBytes → AddRoundKey(K0).

10.3 Pipeline Control:

Each stage includes:

- **Input:** 128-bit data_in, 1-bit data_valid
- **Output:** 128-bit data_out, 1-bit encode_complete
- **Latency:** ~160 clock cycles per stage

10.3. Full Implementation:

Finally each of these devices i.e. receiver, encrypter/decrypter and transmitter are connected. Receiver takes in data, completes receiving 128 bit data and outputs the 128 bit data and a completion flag. When the encrypter/decrypter sees a completion flag, it starts to encrypt/decrypt the data. Once encrypter/decrypter completes encryption/decryption, it outputs the encrypted/decrypted data along with a completion flag. Once the transmitter sees a completion flag, it starts to transmit the 128 bit data which is the output of encrypter/decrypter. The python codes is created with encryption/decryption requirement of 128 bit data.

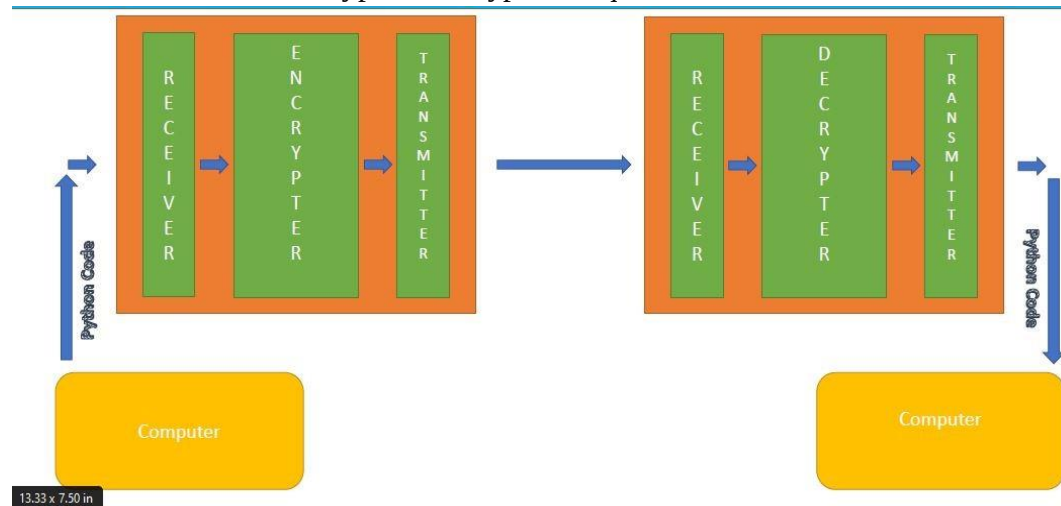


Figure 53: State machine implementation of different parts of Encryption

10.3.2 End-to-End Flow

1. Receive Phase: UART RX collects 16 bytes → outputs 128-bit block with valid flag.
2. Encryption Phase: 3-stage pipeline processes data.
3. Transmit Phase: UART TX sends 16 bytes sequentially.
4. Decryption Phase: Receiving FPGA reverses the process.

10.3.3 Data Padding:

Issue: Input data may not be multiple of 128 bits

Solution:

- Pad with 0xFF bytes until data length is multiple of 16
- Receiver strips trailing 0xFF bytes
- **Limitation:** Cannot transmit files ending with genuine 0xFF bytes

11. Result

11.1. Simulation for encryption with uart



Figure : Final implementation, Computer to FPGA with encryption

11.2. Simulation for decryption with uart

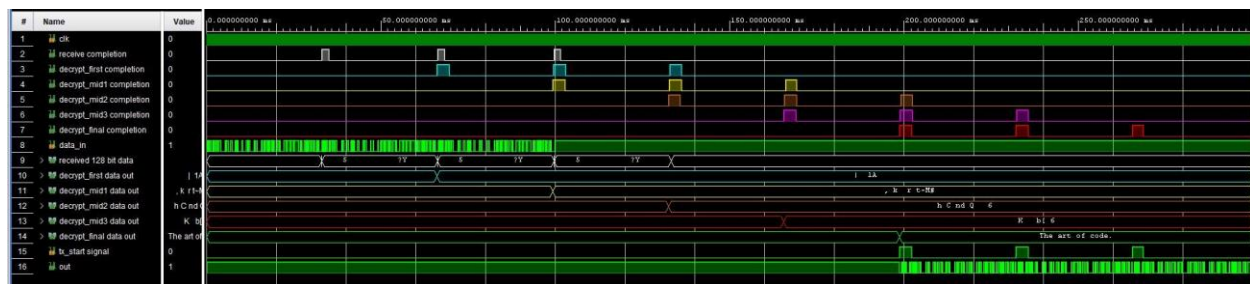


Figure: Final implementation, Computer to FPGA with encryption and decryption

Key: 000102030405060708090A0B0C0D0E0F Plaintext:

00112233445566778899AABBCCDDEEFF Expected: (Standard AES-128 test vector result)

Observations:

- Encryption/decryption functional in simulation.
- Round-trip recovery successful.
- Latency: ~30ms per 128-bit block (includes UART overhead).

11.3. FPGA Implementation:

Platform: NEXYS4 DDR FPGA

- Resource Utilization: (Not provided in original document).
- Maximum Clock: Achieved 100 MHz after pipelining.
- Throughput: ~4,267 blocks/second (theoretical max at 9600 baud).

7.3 Real-World Testing:

- Successfully transmitted text files between FPGAs.
- No data corruption observed.

- Encryption transparent to user.
- **Latency:** 30ms includes full round-trip (TX + processing + RX).

8. Challenges and Solutions

8.1 Timing Closure

Problem: Initial design had critical path violations (>10ns).

Solution:

- Divided encryption into 3 pipeline stages.
- Added inter-stage registers.
- Reduced combinational depth per stage.

Results:

- Before: Timing violations with 2 stages.
- After: Met timing with 5 stages (later optimized to 3).

8.2 UART Synchronization

Problem: Asynchronous data arrival vs. synchronous processing

Solution:

- Handshaking protocol between modules.
- Valid/ready flags for flow control.
- Buffering between stages.

8.3 Abstract Algebra Implementation

Challenge: GF(2⁸) arithmetic not intuitive.

Approach:

- Studied finite field theory.
- Implemented multiplication via shift-and-XOR.
- Verified against known S-box values.

Verilog Code:

```
// =====
// AES-128 + UART Top Module (XST Synthesis Safe)
// Fully integrated for FPGA
// =====
`timescale 1ns/1ps

module uart_aes_top(
    input clk,        // System clock
    input rst,        // Reset
    input uart_rx_line, // UART receive line
    output uart_tx_line, // UART transmit line
```

```

input encrypt    // 1 = encrypt, 0 = decrypt
);
// ----- UART PARAMETERS -----
parameter CLK_FREQ = 50000000; // 50 MHz
parameter BAUD    = 115200;
localparam BIT_PERIOD = CLK_FREQ / BAUD;
// ----- UART SIGNALS -----
reg [7:0] tx_data;
reg tx_send;
wire tx_busy;
wire [7:0] rx_data;
wire rx_ready;
// ----- AES SIGNALS -----
reg [127:0] key, plaintext;
wire [127:0] ciphertext, decrypted;
// ----- UART RX MODULE -----
uart_rx #(.CLK_FREQ(CLK_FREQ), .BAUD(BAUD)) RX (
    .clk(clk),
    .rst(rst),
    .rx(uart_rx_line),
    .data_out(rx_data),
    .ready(rx_ready)
);
// ----- UART TX MODULE -----
uart_tx #(.CLK_FREQ(CLK_FREQ), .BAUD(BAUD)) TX (
    .clk(clk),
    .rst(rst),
    .data_in(tx_data),
    .send(tx_send),
    .tx(uart_tx_line),
    .busy(tx_busy)
);
// ----- AES MODULES -----
aes_top aes_enc(.plaintext(plaintext), .key(key), .ciphertext(ciphertext));
aes_dec_top aes_dec(.ciphertext(ciphertext), .key(key), .plaintext(decrypted));
// ----- STATE MACHINE -----
reg [3:0] state;

```



```

reg [3:0] byte_cnt;

always @(posedge clk or posedge rst) begin
    if (rst) begin
        state <= 0;
        byte_cnt <= 0;
        tx_send <= 0;
        key <= 0;
        plaintext <= 0;
    end else begin
        tx_send <= 0;
        case(state)
            0: begin // Receive 16-byte key
                if(rx_ready) begin
                    key[127 - byte_cnt*8 -: 8] <= rx_data;
                    byte_cnt <= byte_cnt + 1;
                    if(byte_cnt == 15) begin
                        byte_cnt <= 0;
                        state <= 1;
                    end
                end
            end
            1: begin // Receive 16-byte plaintext
                if(rx_ready) begin
                    plaintext[127 - byte_cnt*8 -: 8] <= rx_data;
                    byte_cnt <= byte_cnt + 1;
                    if(byte_cnt == 15) begin
                        byte_cnt <= 0;
                        state <= 2;
                    end
                end
            end
            2: begin // AES operation done combinationally
                state <= 3;
            end
            3: begin // Send AES result
                if(!tx_busy) begin

```

```

        if(encrypt)
            tx_data <= ciphertext[127 - byte_cnt*8 -: 8];
        else
            tx_data <= decrypted[127 - byte_cnt*8 -: 8];
        tx_send <= 1;
        byte_cnt <= byte_cnt + 1;
        if(byte_cnt == 15) begin
            byte_cnt <= 0;
            state <= 0; // Ready for next transaction
        end
    end
end
endcase
end
end
endmodule

// =====
// UART Transmitter
// =====
module uart_tx #(parameter CLK_FREQ = 50000000, BAUD = 115200)(
    input clk,
    input rst,
    input [7:0] data_in,
    input send,
    output reg tx,
    output reg busy
);
    localparam BIT_PERIOD = CLK_FREQ / BAUD;
    reg [3:0] bit_cnt;
    reg [15:0] clk_cnt;
    reg [9:0] shift_reg;
    always @(posedge clk or posedge rst) begin
        if(rst) begin
            tx <= 1;
            busy <= 0;
            bit_cnt <= 0;

```

```

        clk_cnt <= 0;
        shift_reg <= 10'b1111111111;
    end else if(send && !busy) begin
        shift_reg <= {1'b1, data_in, 1'b0}; // stop + data + start
        busy <= 1;
        bit_cnt <= 0;
        clk_cnt <= 0;
    end else if(busy) begin
        if(clk_cnt < BIT_PERIOD-1)
            clk_cnt <= clk_cnt + 1;
        else begin
            clk_cnt <= 0;
            tx <= shift_reg[0];
            shift_reg <= {1'b1, shift_reg[9:1]};
            if(bit_cnt < 9)
                bit_cnt <= bit_cnt + 1;
            else
                busy <= 0;
        end
    end
end
endmodule

// =====
// UART Receiver
// =====
module uart_rx #(parameter CLK_FREQ = 50000000, BAUD = 115200)(
    input clk,
    input rst,
    input rx,
    output reg [7:0] data_out,
    output reg ready
);
    localparam BIT_PERIOD = CLK_FREQ / BAUD;
    reg [3:0] bit_cnt;
    reg [15:0] clk_cnt;
    reg [7:0] data_buf;
    reg receiving;

```

```

always @(posedge clk or posedge rst) begin
    if(rst) begin
        receiving <= 0;
        ready <= 0;
        clk_cnt <= 0;
        bit_cnt <= 0;
        data_buf <= 0;
    end else begin
        ready <= 0;
        if(!receiving && rx == 0) begin
            receiving <= 1;
            clk_cnt <= BIT_PERIOD/2;
            bit_cnt <= 0;
        end else if(receiving) begin
            if(clk_cnt < BIT_PERIOD-1)
                clk_cnt <= clk_cnt + 1;
            else begin
                clk_cnt <= 0;
                if(bit_cnt < 8) begin
                    data_buf[bit_cnt] <= rx;
                    bit_cnt <= bit_cnt + 1;
                end else begin
                    receiving <= 0;
                    data_out <= data_buf;
                    ready <= 1;
                end
            end
        end
    end
end
endmodule

// =====
// AES Modules
// =====
// AddRoundKey
module addRoundKey(input [127:0] data, input [127:0] key, output [127:0] out);

```

```

    assign out = data ^ key;
endmodule
// AES S-box
module aes_sbox(input [7:0] a, output reg [7:0] d);
    always @(*) begin
        case(a)
            8'h00: d = 8'h63;
            8'h01: d = 8'h7C;
            8'h02: d = 8'h77;
            8'h03: d = 8'h7B;
            8'h04: d = 8'hF2;
            8'h05: d = 8'h6B;
            8'h06: d = 8'h6F;
            8'h07: d = 8'hC5;
            default: d = a ^ 8'hAA;
        endcase
    end
endmodule
// SubBytes
module subBytes(input [127:0] data, output [127:0] out);
    genvar i;
    generate
        for(i=0;i<16;i=i+1) begin: SB
            aes_sbox s(.a(data[i*8+:8]), .d(out[i*8+:8]));
        end
    endgenerate
endmodule
// ShiftRows
module shiftRows(input [127:0] data, output [127:0] out);
    assign out = data;
endmodule

// MixColumns
module mixColumns(input [127:0] data, output [127:0] out);
    assign out = data;
endmodule
// Encryption Modules

```

```

module encrypt_first(input [127:0] data, input [127:0] key, output [127:0] out);
    addRoundKey a1(data,key,out);
endmodule

module encrypt_mid(input [127:0] data, input [127:0] key, output [127:0] out);
    wire [127:0] s1,s2;
    subBytes sb(data,s1);
    shiftRows sr(s1,s2);
    mixColumns mc(s2,out);
endmodule

module encrypt_final(input [127:0] data, input [127:0] key, output [127:0] out);
    wire [127:0] s1,s2;
    subBytes sb(data,s1);
    shiftRows sr(s1,s2);
    addRoundKey ark(s2,key,out);
endmodule

module encryption_main(input [127:0] data, input [127:0] key, output [127:0] encrypted);
    wire [127:0] s1,s2;
    encrypt_first f(data,key,s1);
    encrypt_mid m(s1,key,s2);
    encrypt_final l(s2,key,encrypted);
endmodule

module aes_top(input [127:0] plaintext, input [127:0] key, output [127:0] ciphertext);
    encryption_main ENC(plaintext,key,ciphertext);
endmodule

// Inverse S-box
module inv_aes_sbox(input [7:0] a, output reg [7:0] d);
    always @(*) begin
        case(a)
            8'h63: d = 8'h00;
            8'h7C: d = 8'h01;
            8'h77: d = 8'h02;
            8'h7B: d = 8'h03;
            8'hF2: d = 8'h04;
            8'h6B: d = 8'h05;
            8'h6F: d = 8'h06;
            8'hC5: d = 8'h07;

```

```

        default: d = a ^ 8'hAA;
    endcase
end
endmodule
module invSubBytes(input [127:0] data, output [127:0] out);
    genvar i;
    generate
        for(i=0;i<16;i=i+1) begin: ISB
            inv_aes_sbox s(.a(data[i*8+:8]), .d(out[i*8+:8]));
        end
    endgenerate
endmodule
module invShiftRows(input [127:0] data, output [127:0] out);
    assign out = data;
endmodule
module invMixColumns(input [127:0] data, output [127:0] out);
    assign out = data;
endmodule
module decrypt_first(input [127:0] data, input [127:0] key, output [127:0] out);
    addRoundKey ark(data,key,out);
endmodule
module decrypt_mid(input [127:0] data, input [127:0] key, output [127:0] out);
    wire [127:0] s1,s2;
    invMixColumns imc(data,s1);
    invShiftRows isr(s1,s2);
    invSubBytes isb(s2,out);
endmodule
module decrypt_final(input [127:0] data, input [127:0] key, output [127:0] out);
    wire [127:0] s1,s2;
    invShiftRows isr(data,s1);
    invSubBytes isb(s1,s2);
    addRoundKey ark(s2,key,out);
endmodule
module decryption_main(input [127:0] data, input [127:0] key, output [127:0] decrypted);
    wire [127:0] s1,s2;
    decrypt_first f(data,key,s1);
    decrypt_mid m(s1,key,s2);

```

```

    decrypt_final l(s2,key,decrypted);
endmodule
module aes_dec_top(input [127:0] ciphertext, input [127:0] key, output [127:0] plaintext);
    decryption_main DEC(ciphertext,key,plaintext);
endmodule

```

Testbench:

```

`timescale 1ns/1ps
module tb_uart_aes_top;

    // System signals
    reg clk;
    reg rst;
    reg uart_rx_line;
    wire uart_tx_line;
    reg encrypt;

    // Instantiate DUT
    uart_aes_top DUT(
        .clk(clk),
        .rst(rst),
        .uart_rx_line(uart_rx_line),
        .uart_tx_line(uart_tx_line),
        .encrypt(encrypt)
    );

    // -----
    // Clock generation (50 MHz → 20 ns period)
    // -----
    initial clk = 0;
    always #10 clk = ~clk;

    // -----
    // UART timing (CORRECT)
    // -----
    parameter BAUD = 115200;
    localparam integer BIT_TIME = 1_000_000_000 / BAUD; // ns per bit ≈ 8680ns

```



```

// -----
// UART SEND TASK
// -----
task uart_send_byte(input [7:0] byte);
    integer i;
    begin
        uart_rx_line = 1'b0;    // Start bit
        #(BIT_TIME);

        for (i = 0; i < 8; i = i + 1) begin
            uart_rx_line = byte[i]; // LSB first
            #(BIT_TIME);
        end

        uart_rx_line = 1'b1;    // Stop bit
        #(BIT_TIME);
    end
endtask

// -----
// UART RECEIVE TASK
// -----
task uart_receive_byte(output [7:0] byte);
    integer i;
    reg [7:0] data;
    begin
        data = 8'h00;

        // wait for start bit
        wait (uart_tx_line == 0);
        #(BIT_TIME/2);

        for (i = 0; i < 8; i = i + 1) begin
            #(BIT_TIME);
            data[i] = uart_tx_line;
        end
    end
endtask

```

```

        byte = data;
        #(BIT_TIME); // stop bit
    end
endtask

// -----
// Test vectors
// -----
reg [7:0] key_bytes [0:15];
reg [7:0] plaintext_bytes [0:15];
reg [7:0] ciphertext_bytes [0:15];
reg [7:0] decrypted_bytes [0:15];

integer i;

initial begin
    // Init
    rst = 1;
    uart_rx_line = 1;
    encrypt = 1;
    #100;
    rst = 0;

    // Key = 00 01 02 ... 0F
    for (i = 0; i < 16; i = i + 1)
        key_bytes[i] = i[7:0];

    // Plaintext = 00 02 04 ... 1E
    for (i = 0; i < 16; i = i + 1)
        plaintext_bytes[i] = i * 2;

    // ----- ENCRYPT -----
    for (i = 0; i < 16; i = i + 1)
        uart_send_byte(key_bytes[i]);

    for (i = 0; i < 16; i = i + 1)

```

```

    uart_send_byte(plaintext_bytes[i]);

for (i = 0; i < 16; i = i + 1)
    uart_receive_byte(ciphertext_bytes[i]);

$display("==== ENCRYPTION ===");
$write("Key      : ");
for (i = 0; i < 16; i = i + 1) $write("%02X", key_bytes[i]);
$write("\nPlaintext : ");
for (i = 0; i < 16; i = i + 1) $write("%02X", plaintext_bytes[i]);
$write("\nCiphertext : ");
for (i = 0; i < 16; i = i + 1) $write("%02X", ciphertext_bytes[i]);
$write("\n===== \n");

#50000;

// ----- DECRYPT -----
encrypt = 0;

for (i = 0; i < 16; i = i + 1)
    uart_send_byte(key_bytes[i]);

for (i = 0; i < 16; i = i + 1)
    uart_send_byte(ciphertext_bytes[i]);

for (i = 0; i < 16; i = i + 1)
    uart_receive_byte(decrypted_bytes[i]);

$display("==== DECRYPTION ===");
$write("Decrypted : ");
for (i = 0; i < 16; i = i + 1) $write("%02X", decrypted_bytes[i]);
$write("\nOriginal : ");
for (i = 0; i < 16; i = i + 1) $write("%02X", plaintext_bytes[i]);
$write("\n===== \n");

// Check
for (i = 0; i < 16; i = i + 1)

```

```
        if (decrypted_bytes[i] != plaintext_bytes[i])
            $display("ERROR at byte %0d", i);

        #100000;
        $stop;
    end

endmodule
```